

Lista de Exercícios: Git, GitHub e GitFlow

Contexto do Projeto

Você foi contratado para desenvolver o site da **Lanchonete do Bairro**. O projeto consiste em uma página web simples com os arquivos `index.html` (página principal), `style.css` (estilos) e `cardapio.js` (lógica do cardápio). Além disso, haverá um arquivo `README.md` com as instruções do projeto. Todos os exercícios a seguir são baseados nesse projeto. **Execute os comandos no terminal da sua máquina, dentro de uma pasta chamada lanchonete-web.**

Referência Visual: Wireframes do Projeto

Use os wireframes abaixo como guia para criar os arquivos HTML do projeto.

`index.html` — Página Principal



`promocoos.html` — Página de Promoções



Parte 1: Fundamentos do Git

Questão 1: Inicializando o Repositório

Crie a pasta do projeto e inicialize o repositório Git. Em seguida, crie os arquivos iniciais do projeto.

- a) Execute os comandos abaixo e copie a saída do terminal:

```
mkdir lanchonete-web  
cd lanchonete-web  
git init
```

- b) Crie manualmente os arquivos `index.html`, `style.css`, `cardapio.js` e `README.md` dentro da pasta (podem estar vazios por enquanto).
- c) Execute `git status` e explique o que significa o status “*Untracked files*” exibido para esses arquivos.

Questão 2: Primeiro Commit: Preparando e Salvando

Com os arquivos criados, registre a primeira versão do projeto no histórico do Git.

- a) Adicione o arquivo `README.md` individualmente à *Staging Area* usando `git add README.md` e execute `git status`. O que mudou?
- b) Adicione todos os demais arquivos de uma vez com `git add .`
- c) Realize o commit inicial com a mensagem: `feat: estrutura inicial do projeto`. Anote o hash curto exibido (ex: `a1b2c3d`).
- d) Execute `git log --oneline` e descreva o que é exibido.

Questão 3: Modificando e Rastreando Arquivos

Edite o arquivo `index.html` adicionando uma estrutura HTML básica com o título “Lanchonete do Bairro”. Em seguida:

- a) Execute `git status` e explique a diferença entre um arquivo *Untracked* e um arquivo *Modified*.
- b) Execute `git diff` antes de fazer o `git add`. O que esse comando mostra?
- c) Faça o `git add` e o `git commit` com a mensagem `feat: adiciona estrutura basica do HTML`.

Questão 4: Histórico de Versões com git log

Faça pelo menos mais duas alterações em arquivos diferentes (ex: adicionar conteúdo ao `style.css` e ao `cardapio.js`), realizando um commit para cada uma. Depois:

- a) Execute `git log` (versão completa) e identifique: o hash completo, o autor, a data e a mensagem de cada commit.
- b) Execute `git log --oneline --graph` e descreva o que o símbolo `*` representa na saída.
- c) Qual a importância de escrever mensagens de commit claras e descritivas em um projeto profissional?

Questão 5: Trabalhando com Branches: Nova Funcionalidade

A lanchonete quer testar uma nova página de “Promoções” antes de publicar no site oficial. Crie uma branch para isso.

- a) Crie e já mude para a branch com: `git checkout -b feature/pagina-promocoes`
-

- b) Crie um arquivo `promocoas.html` com algum conteúdo e faça o commit dele na nova branch.
- c) Execute `git branch` e explique o que o asterisco (*) ao lado de um nome indica.
- d) Volte para a branch `main` com `git checkout main` e verifique se o arquivo `promocoas.html` está visível. O que aconteceu com ele e por quê?

Questão 6: Integrando Branches com git merge

A página de promoções foi aprovada! Hora de integrá-la ao site oficial.

- a) Estando na branch `main`, execute `git merge feature/pagina-promocoas`.
- b) Copie a saída do terminal e identifique qual estratégia de merge o Git utilizou.
- c) Execute `git log --oneline --graph` e descreva visualmente o que mudou no histórico após o merge.
- d) Após o merge, delete a branch de feature com `git branch -d feature/pagina-promocoas`. Por que é uma boa prática deletar branches concluídas?

Questão 7: Desfazendo Erros com git revert

Imagine que o último commit introduziu um erro no `style.css` (ex: cor de fundo errada). Precisamos desfazê-lo.

- a) Faça um commit com uma alteração proposital “errada” no `style.css` (ex: adicionar `background: pink`).
- b) Usando `git log --oneline`, identifique o hash desse commit ruim.
- c) Execute `git revert <hash>` para desfazê-lo. Qual a diferença entre `git revert` e `git reset`? Por que o `revert` é mais seguro em projetos colaborativos?

Questão 8: Repositório Remoto com GitHub

Leve o projeto local para a nuvem usando o GitHub.

- a) Crie um repositório **público** no GitHub chamado `lanchonete-web` (sem inicializar com README).
- b) Conecte o repositório local ao remoto:

```
git remote add origin https://github.com/<seu-usuario>/
lanchonete-web.git
git push -u origin main
```

- c) Acesse o repositório no GitHub pelo navegador e tire um print da tela com o histórico de commits. O que significa a flag `-u` no `git push`?
- d) Faça uma pequena alteração diretamente pelo editor do GitHub (no `README.md`) e depois execute `git pull` localmente. O que o `git pull` fez?

Parte 2: GitFlow — Fluxo Profissional de Desenvolvimento

Questão 9: Inicializando o GitFlow

Com o projeto da lanchonete já no GitHub, vamos aplicar a metodologia GitFlow.

- a) Instale o plugin GitFlow (se necessário: `sudo apt install git-flow` no Linux).
- b) Execute `git flow init` dentro da pasta `lanchonete-web`. Pressione **Enter** em todas as perguntas para aceitar os valores padrão. Copie toda a saída do comando.
- c) Execute `git branch` e liste todas as branches criadas. Qual branch foi criada além da `main`? Para que ela serve?
- d) Faça o push da branch `develop` para o GitHub: `git push -u origin develop`.

Questão 10: Ciclo de Feature: Novo Cardápio Online

A lanchonete quer um cardápio interativo. Aplique o fluxo de *feature* corretamente.

- a) Inicie a feature com: `git flow feature start cardapio-interativo`. Em qual branch você está agora?
- b) Edite o arquivo `cardapio.js` adicionando um array de objetos com pelo menos 3 itens do cardápio (ex: nome, preço, categoria). Faça dois commits com mensagens descritivas durante o desenvolvimento.
- c) Finalize a feature com `git flow feature finish cardapio-interativo`. O que o comando fez automaticamente? Para qual branch o código foi enviado?
- d) Verifique com `git log --oneline --graph` e desenhe (ou descreva) o fluxo que ocorreu.

Questão 11: Ciclo de Release: Preparando a Versão 1.0

As funcionalidades da primeira versão estão prontas. Hora de preparar o lançamento oficial.

- a) Inicie a release com: `git flow release start 1.0.0`. Em qual branch o fluxo de release é criado? A partir de qual branch ela nasce?
- b) Na branch de release, faça apenas ajustes de preparação para o lançamento: atualize o `README.md` com instruções de uso e corrija qualquer texto no `index.html`. Faça um commit com a mensagem chore: `prepara release v1.0.0`.
- c) Finalize a release com `git flow release finish 1.0.0`. Explique o que ocorreu com as branches `main` e `develop` após esse comando.
- d) Execute `git tag` e verifique se a tag `1.0.0` foi criada. Qual é a função de uma tag no contexto do GitFlow?

Questão 12: Ciclo de Hotfix: Corrigindo Bug em Produção

Após o lançamento, um cliente reportou que o título da página está escrito errado no `index.html`. Precisa ser corrigido *urgentemente* em produção!

- a) Inicie o hotfix a partir da `main`: `git flow hotfix start correcao-titulo`. Explique por que o hotfix nasce da `main` e não da `develop`.
- b) Corrija o erro no `index.html` e faça o commit com a mensagem fix: `corrige titulo da pagina principal`.

- c) Finalize o hotfix com `git flow hotfix finish correcao-titulo`. Após a finalização, execute `git log --oneline --all --graph` e identifique para quais branches o hotfix foi mesclado.
- d) Qual seria o número de versão correto após esse hotfix, seguindo o padrão do Versionamento Semântico (SemVer)? Justifique.

Questão 13: Reflexão Final: GitFlow na Prática

Após completar todos os exercícios, reflita sobre o fluxo de trabalho aplicado.

- a) Desenhe (à mão ou digitalmente) o diagrama completo do histórico de branches do projeto da lanchonete, representando: as branches `main` e `develop`, a *feature*, a *release* e o *hotfix*, com as setas de merge entre elas.
- b) Em que tipo de projeto o GitFlow seria **mais indicado**? E em qual seria **desnecessariamente complexo**? Justifique com base nas suas experiências durante os exercícios.
- c) Compare o histórico de branches de um projeto **sem** GitFlow (como foi nos exercícios 1–8) com um projeto **com** GitFlow (exercícios 9–13). Cite uma vantagem e uma desvantagem percebidas.

Instruções de Entrega

A entrega desta lista é feita **pelo próprio repositório GitHub** criado durante os exercícios, juntamente com um arquivo PDF com as respostas conceituais. Envie no campo de entrega do Google Classroom **dois itens**: um link do repositório público e um arquivo PDF com as respostas conceituais. Pode colocar os arquivos dentro de um arquivo zipado:

1. **Link do Repositório Público:** A URL completa do repositório `lanchonete-web` no GitHub
(exemplo: `https://github.com/seu-usuario/lanchonete-web`)
2. **Arquivo PDF com as Respostas Conceituais:** Um documento contendo as respostas textuais das questões teóricas (itens que pedem explicação, diferenciação ou reflexão). Esse documento pode ser gerado como um arquivo no próprio repositório (`respostas.md` ou `respostas.pdf`).

Critérios de Avaliação (Rubrica)

Critério	O que será avaliado	Como verificar
Qualidade dos Comits	Mensagens claras, descritivas e no formato correto	<code>git log</code> no repositório
Uso de Branches	Criação, uso isolado e deleção após o merge	Aba “Insights → Network” do GitHub
Fluxo GitFlow	Branches <code>feature/</code> , <code>release/</code> e <code>hotfix/</code> corretamente aplicadas	Histórico de branches e tags
Tags de Versão	Presença das tags 1.0.0 e 1.1.1 (ou superior)	Aba “Releases / Tags” do GitHub
Repositório Remoto	Repo público com todos os arquivos e histórico completo	Acesso direto à URL enviada
Respostas Conceituais	Clareza, corretude e profundidade das explicações textuais	Arquivo PDF ou <code>respostas.md</code>